



NAME – SUBHASH KUMAR AND SHUBHAM SINGH

ROLL-NUMBER -; 202410116100211, 202410116100204

SECTION – D

GROUP – 10

SUBJECT- AI

```
# Step 1: Import libraries  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
import shap

import warnings

warnings.filterwarnings("ignore")


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve,
precision_recall_curve


from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout, BatchNormalization

from tensorflow.keras.callbacks import EarlyStopping

from tensorflow.keras.regularizers import l2


# Step 2: Load dataset

df = pd.read_csv('/content/heart.csv') # Adjust path if needed

print("First 5 rows of the dataset:")

print(df.head())


# Step 3: Visualizations

plt.figure(figsize=(10, 8))

sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')

plt.title("Correlation Heatmap of Features")

plt.show()


sns.countplot(data=df, x='target', palette='Set2')

plt.title("Distribution of Heart Disease (Target Variable)")

plt.show()


# Step 4: Feature and target separation

X = df.drop('target', axis=1)
```

```
y = df['target']
```

```
# Step 5: Train-test split and normalization
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# Step 6: Build ANN model
```

```
model = Sequential([
```

```
    Dense(64, activation='relu', kernel_regularizer=l2(0.001), input_shape=(X_train.shape[1],)),
```

```
    BatchNormalization(),
```

```
    Dropout(0.4),
```

```
    Dense(32, activation='relu', kernel_regularizer=l2(0.001)),
```

```
    BatchNormalization(),
```

```
    Dropout(0.3),
```

```
    Dense(16, activation='relu', kernel_regularizer=l2(0.001)),
```

```
    Dropout(0.2),
```

```
    Dense(1, activation='sigmoid')
```

```
])
```

```
model.compile(optimizer='nadam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
# Step 7: Train model
```

```
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```

```
history = model.fit(X_train_scaled, y_train, validation_split=0.2, epochs=100, batch_size=32,  
callbacks=[early_stop])
```

```
# Step 8: Plot accuracy/loss
```

```
plt.figure(figsize=(10, 4))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.legend()
plt.title('Model Accuracy')
```

```
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.title('Model Loss')
plt.show()
```

```
# Step 9: Evaluate
```

```
y_pred_prob = model.predict(X_test_scaled).ravel()
y_pred = (y_pred_prob > 0.5).astype(int)
```

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
# ROC Curve
```

```
# ROC Curve
```

```
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
auc_score = roc_auc_score(y_test, y_pred_prob)
```

```
plt.plot(fpr, tpr, label=f"AUC = {auc_score:.2f}")

plt.plot([0, 1], [0, 1], 'k--')

plt.title("ROC Curve")

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

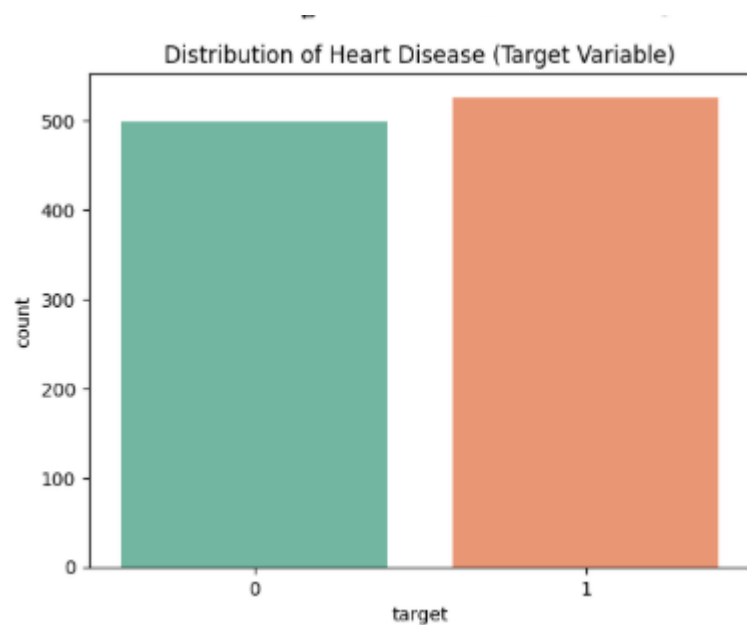
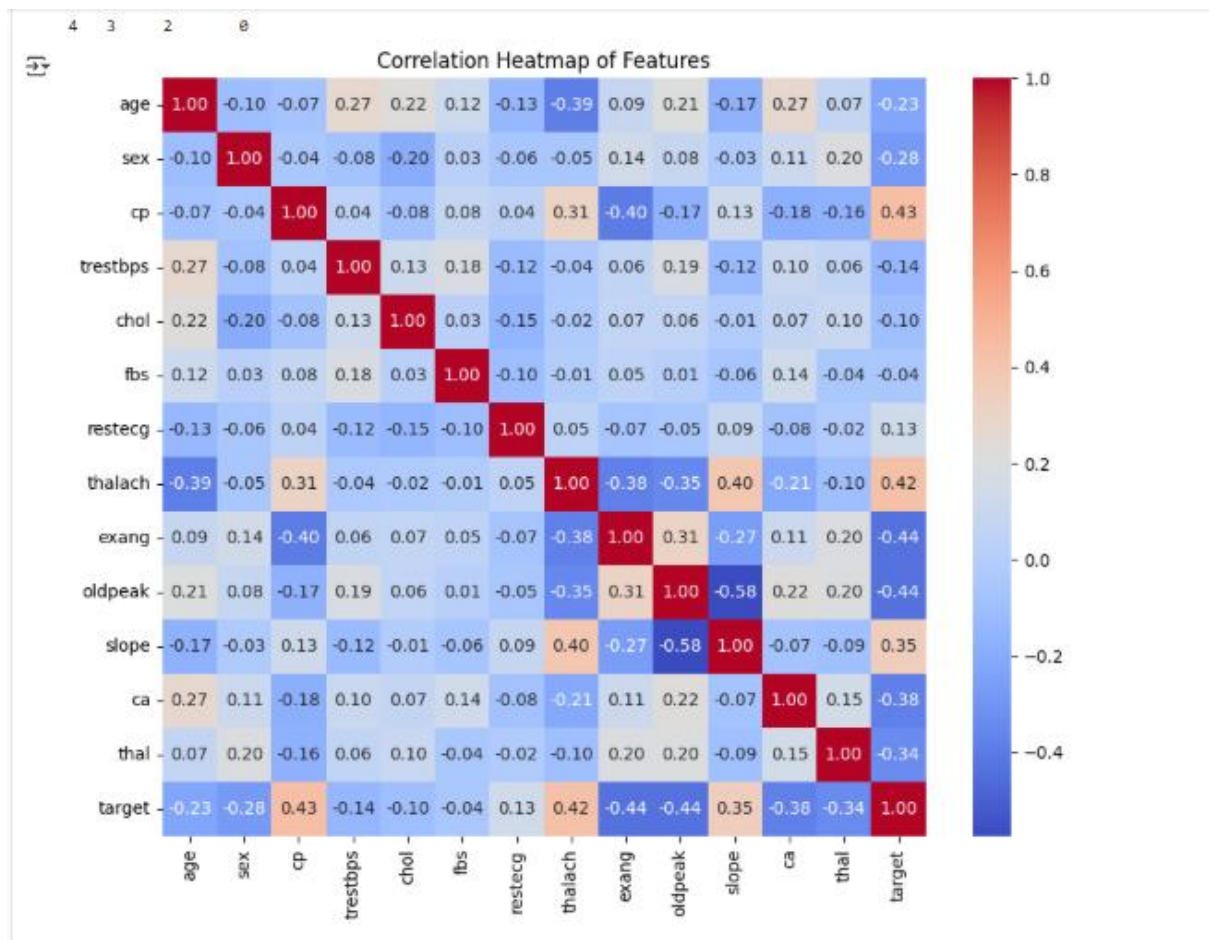
plt.legend()

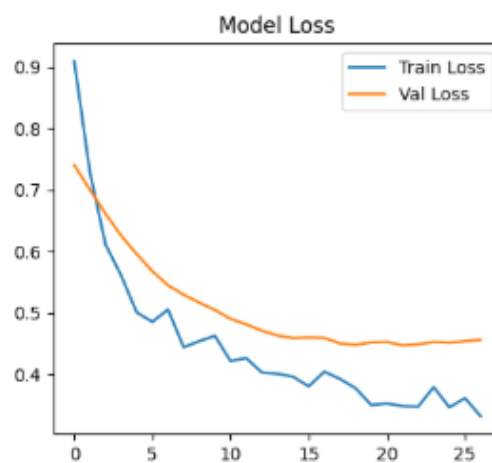
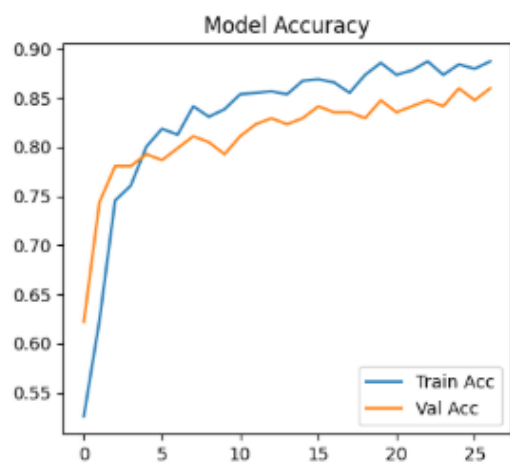
plt.show()
```

First 5 rows of the dataset:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	\
0	52	1	0	125	212	0	1	168	0	1.0	2	
1	53	1	0	140	203	1	0	155	1	3.1	0	
2	70	1	0	145	174	0	1	125	1	2.6	0	
3	61	1	0	148	203	0	1	161	0	0.0	2	
4	62	0	0	138	294	1	1	106	0	1.9	1	

	ca	thal	target
0	2	3	0
1	0	3	0
2	0	3	0
3	1	3	0
4	3	2	0





7/7 0s 19ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.74	0.80	102
1	0.78	0.90	0.83	103
accuracy			0.82	205
macro avg	0.83	0.82	0.82	205
weighted avg	0.83	0.82	0.82	205

